

# 世界先進 AI 推論效能驗證 POC

RTX PRO 6000 × aiDAPTIV+

以 TTFT 與 Tokens/sec 驗證 middleware 在大型模型、高併發與長文本情境下的效能優勢。

# 01 POC 目的與測試架構

同一套硬體、同一組模型與參數，只比較 aiDAPTIV+ 開啟與關閉

## 驗測目的

- 驗證 aiDAPTIV+ 對首字延遲 TTFT 的改善。
- 驗證高併發下 Tokens/sec 的效能曲線與服務承載能力。

### 未啟用 aiDAPTIV+

RTX PRO 6000 4-GPU Server

Framework : vLLM

aiDAPTIV+ : 關閉

### 啟用 aiDAPTIV+

RTX PRO 6000 4-GPU Server

Framework : vLLM

aiDAPTIV+ : 開啟

## 02 aiDAPTIV+ 系統效能驗測

實驗配置

### 設備資訊

|     |                              |
|-----|------------------------------|
| GPU | NVIDIA RTX PRO 6000 96GB × 4 |
|-----|------------------------------|

|     |                        |
|-----|------------------------|
| SSD | PASCARI AI100E 2TB × 4 |
|-----|------------------------|

|           |      |
|-----------|------|
| Framework | vLLM |
|-----------|------|

|              |                    |
|--------------|--------------------|
| Acceleration | aiDAPTIV NXUN303.A |
|--------------|--------------------|

|                 |                       |
|-----------------|-----------------------|
| Evaluation Tool | NVIDIA GenAI-Perf CLI |
|-----------------|-----------------------|

### 實驗參數

|    |                            |
|----|----------------------------|
| 模型 | GPT-OSS-120B、Llama 3.3 70B |
|----|----------------------------|

|    |      |
|----|------|
| 精度 | FP16 |
|----|------|

|              |                         |
|--------------|-------------------------|
| Input Tokens | 1K / 2K / 4K / 8K / 16K |
|--------------|-------------------------|

|               |         |
|---------------|---------|
| Output Tokens | 1K / 2K |
|---------------|---------|

|             |                              |
|-------------|------------------------------|
| Concurrency | 1 / 2 / 4 / 8 / 16 / 32 / 64 |
|-------------|------------------------------|

|      |        |
|------|--------|
| 測試次數 | 每組 3 次 |
|------|--------|

## 03 測試矩陣

兩個模型 × 五組輸入 × 兩組輸出 × 七組併發

完整組合

每個測試組合：Run 3 次（冷啟動 × 1 + 熱啟動 × 2）

單一 Model：5 Input × 2 Output × 7 Concurrency × 3 Runs = 210 次；with / without aiDAPTIV+ = 420 次；2 Models = 840 次

| 測試模型          | Input Tokens            | Output Tokens | Concurrency                  |
|---------------|-------------------------|---------------|------------------------------|
| GPT-OSS-120B  | 1K / 2K / 4K / 8K / 16K | 1K / 2K       | 1 / 2 / 4 / 8 / 16 / 32 / 64 |
| Llama 3.3 70B | 1K / 2K / 4K / 8K / 16K | 1K / 2K       | 1 / 2 / 4 / 8 / 16 / 32 / 64 |



## 04 核心評估指標 (KPI)

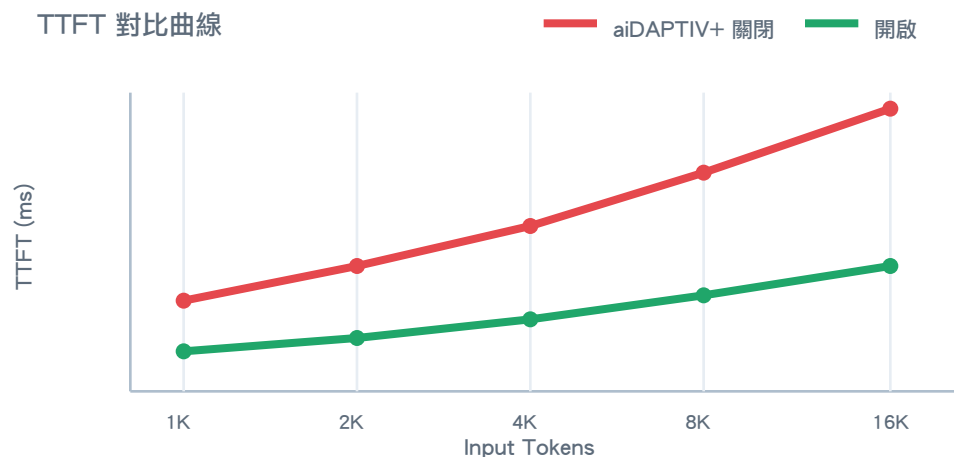
以兩條曲線直接比較 aiDAPTIV+ 開啟與關閉

### TTFT | 首字延遲

延遲降低超過 50%

相同 Input Tokens 下，比較第一個 Token 回傳時間。

示意圖 | 非實測數據

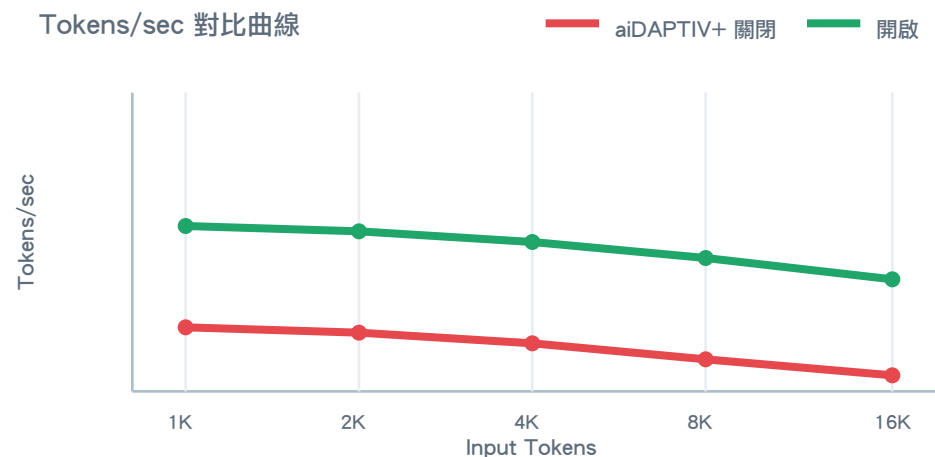


### Tokens/sec | 生成速度

生成速度提升超過 50%

相同 Input Tokens 下，比較每秒生成 Token 數。

示意圖 | 非實測數據



# 05 執行流程與交付結果

確認 POC 方案、執行 Benchmark 並完成效能驗證

01

## POC 方案確認

確認設備、模型與測試參數

02

## POC Kickoff

經由會議確認雙方人員協作方式

03

## Benchmark

執行關閉／開啟對比測試

04

## 效能驗證

彙整 TTFT 與 Tokens/sec KPI

### 交付結果

● TTFT 比較曲線

● Tokens/sec 比較曲線

● 測試原始數據

● 導入配置建議

# 06 Benchmark (Llama 3.3 70B 為例)

環境確認與 aiDAPTIV+ 容器啟動

## 第一步 | 確認狀態

```
$ sudo docker ps -a  
$ nvidia-smi
```

## 第二步 | 啟動容器

```
$ sudo docker run --name vllm_llama70b_test --gpus all -it \  
  --ipc=host --privileged=true \  
  --ulimit memlock=-1 --ulimit stack=67108864 \  
  -e CUDA_VISIBLE_DEVICES=0,1,2,3 \  
  -p 8002:8002 \  
  -v /home/tpiuser/Desktop/llm:/app \  
  -v /mnt/nvme0:/mnt/nvme0 \  
  -v /dev/mapper:/dev/mapper \  
  aidaptiv:VNXUN_3_03_AA
```

# 06 Benchmark (續)

啟動 without server、執行測試，再切換 with aiDAPTIV+

## 第三步 | 啟動 server (without)

```
$ cd /home/root/aiDAPTIV2/script
$ VLLM_USE_V1=1; export \
  VLLM_WORKER_MULTIPROC_METHOD=spawn
$ nohup python3 vllm_api_server.py \
  --model /app/models/Llama-3.3-70B-Instruct \
  --tensor-parallel-size 4 \
  --port 8002 --max-model-len 20000 \
  --gpu-memory-utilization 0.85 \
  --nvme-path /mnt/nvme0/ --dram-kv-offload-gb 0 \
  --disable-gpu-reuse --ssd-kv-offload-gb 500 \
  --no-resume-kv-cache --disable-kv-cache-offload \
  --disable-custom-all-reduce \
  > /app/logs/llama3.3_70B/server_without.log 2>&1 &
```

## 第四步 | 執行 benchmark

```
$ cd /app/benchmarks/core
$ mkdir -p /app/logs/llama3.3_70B
$ nohup bash \
  /app/benchmarks/models/llama3.3_70B/concurrent_random_llama.sh \
  > /app/logs/llama3.3_70B/origin_llama.log 2>&1 &
```

```
# ■■■■■■ 60 ■■
```

```
$ grep -c "Successful requests" \
  /app/logs/llama3.3_70B/origin_llama.log
```

```
■■ ■■ ■■ ■■
```

```
$ nohup bash \
  /app/benchmarks/models/llama3.3_70B/concurrent_random_llama.sh \
  > /app/logs/llama3.3_70B/aidaptiv_llama.log 2>&1 &
```

# 06 Benchmark | 參數說明

依 Pro 6000 4-GPU 環境設定，執行前應再次確認實際路徑與容量

## 容器與運算參數

**CUDA\_VISIBLE\_DEVICES=0,1,2,3**

指定本次使用的 4 張 GPU。

**--model**

指定模型目錄或模型名稱。

**--tensor-parallel-size 4**

將模型切分至 4 張 GPU 執行。

**--port 8002**

設定 vLLM API 服務連接埠。

**--max-model-len 20000**

設定模型最大 Context 長度。

**--gpu-memory-utilization 0.85**

允許 vLLM 使用 85% GPU 顯存。

## aiDAPTIV+ 與 KV Cache 參數

**--nvme-path /mnt/nvme0/**

指定 aiDAPTIV+ SSD 掛載路徑。

**--dram-kv-offload-gb 0**

DRAM KV Cache 卸載容量設為 0GB。

**--ssd-kv-offload-gb 500**

配置 500GB SSD 空間供 KV Cache 卸載。

**--disable-gpu-reuse**

停用 GPU Cache 重複使用機制。

**--no-resume-kv-cache**

啟動時不載入先前保存的 KV Cache。

**--disable-kv-cache-offload**

關閉 KV Cache 卸載，用於 without 基準組。

**remove --disable-kv-cache-offload**

開啟 KV Cache 卸載，用於 with aiDAPTIV+ 組。

# 06 Benchmark | with / without Commands

兩組使用相同模型與參數，僅切換 KV Cache Offload

## WITHOUT | 純顯存基準組

```
$ python3 vllm_api_server.py \
  --model /app/models/Llama-3.3-70B-Instruct \
  --tensor-parallel-size 4 \
  --port 8002 --max-model-len 20000 \
  --gpu-memory-utilization 0.85 \
  --nvme-path /mnt/nvme0/ --dram-kv-offload-gb 0 \
  --ssd-kv-offload-gb 500 --no-resume-kv-cache \
  --disable-gpu-reuse --disable-kv-cache-offload \
  --disable-custom-all-reduce
```

## WITH | aiDAPTIV+ 測試組

```
$ python3 vllm_api_server.py \
  --model /app/models/Llama-3.3-70B-Instruct \
  --tensor-parallel-size 4 \
  --port 8002 --max-model-len 20000 \
  --gpu-memory-utilization 0.85 \
  --nvme-path /mnt/nvme0/ --dram-kv-offload-gb 0 \
  --ssd-kv-offload-gb 500 --no-resume-kv-cache \
  --disable-gpu-reuse \
  --disable-custom-all-reduce
```

### 核心差異

without：加入 `--disable-kv-cache-offload`，KV Cache 僅使用 GPU 顯存，作為基準。

with：移除此參數，啟用 SSD KV Cache Offload，提升併發與長 Context 承載；延遲受 SSD I/O 影響。

# 07 WebUI Testing Reference

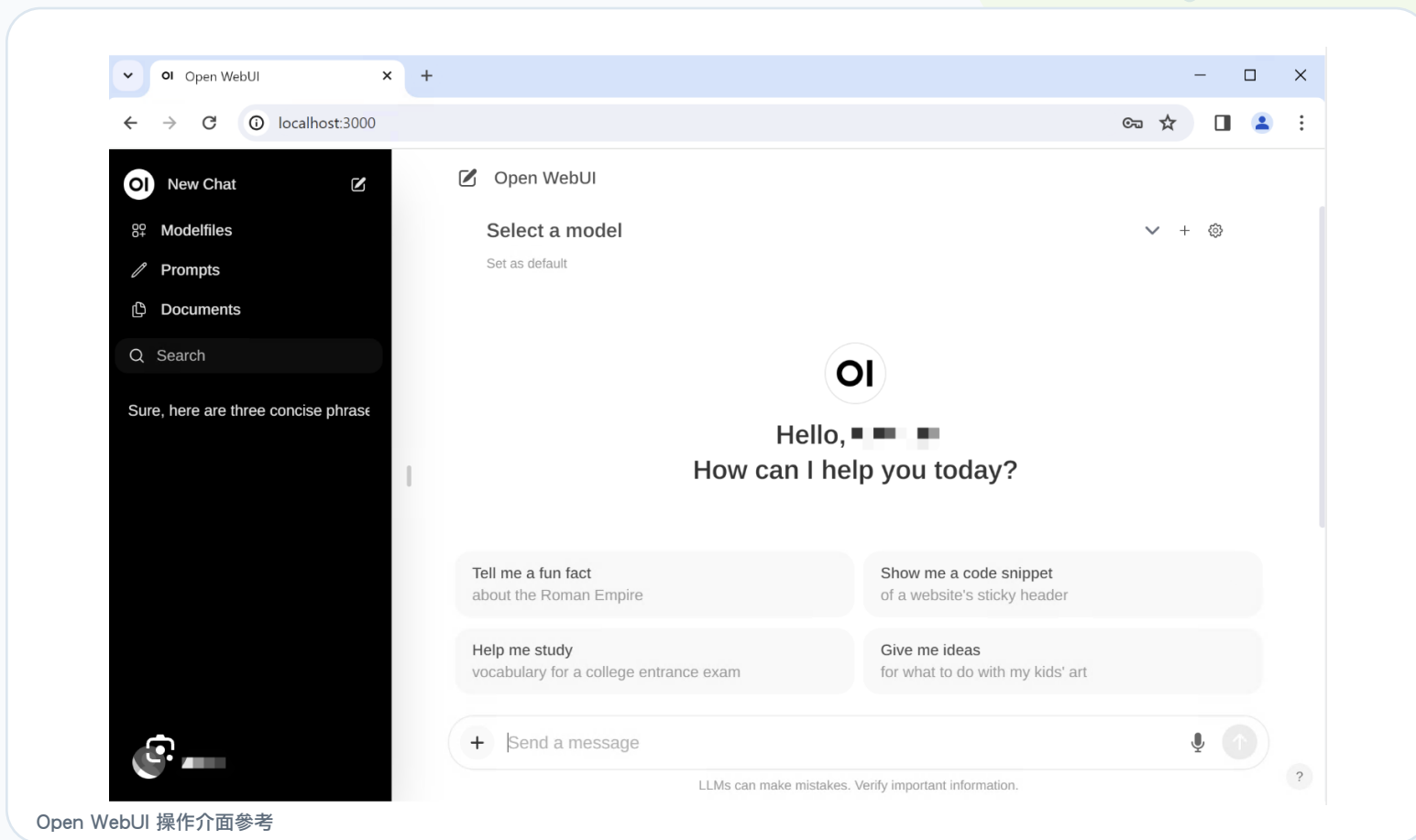
以 Open WebUI 連接 vLLM，進行瀏覽器實際操作與使用體感確認

## 操作與驗證

### OPEN WEBUI

- 模型切換與對話測試
- 長文本輸入
- 首字等待時間
- 生成流暢度
- 錯誤與中斷紀錄

連線介面：localhost:3000後端：vLLM API



Open WebUI 操作介面參考

# 08 Next Step

依客戶實際需求完成系統規格與機台配置

## 需求導向配置

POC 完成後，將與世界先進進行討論，並評估所需的系統架構與硬體規格。

01

### 需求確認

模型／精度

使用人數／應用情境

02

### 效能基準

TTFT／Tokens/sec

Concurrency／Context

03

### 容量換算

GPU／SSD 數量

單機或多機架構

04

### 方案配置

機台規格／數量

擴充與維運方式